Prueba de desempeño – Módulo 6.2 Nest JS

Caso de uso:

La empresa de transporte **FleetCore** quiere implementar un sistema para gestionar el mantenimiento de su flota de vehículos.

Actualmente, el control de los mantenimientos preventivos y correctivos se lleva a cabo mediante hojas de cálculo y registros manuales, lo que genera duplicidad de información, dificultades para conocer la disponibilidad de los vehículos, retrasos en la programación de mantenimientos y poca trazabilidad sobre las intervenciones realizadas.

Objetivo:

Bajo tu rol como desarrollador backend, deberás construir una **API REST con NestJS, TypeScript, JWT, TypeOrm, PostgreSQL** que permita administrar el ciclo de vida de los mantenimientos realizados sobre los vehículos de la empresa.

El sistema debe permitir:

- Registrar y administrar los **vehículos** pertenecientes a la flota.
- Registrar y administrar los **mecánicos** responsables de realizar los mantenimientos.
- Registrar y administrar los **talleres** donde se llevarán a cabo las intervenciones.
- Crear mantenimientos **preventivos y correctivos** asociados a un vehículo.
- Asignar un **mecánico y un taller** a cada mantenimiento.
- Controlar el estado de cada mantenimiento: **programado, en proceso, completado o cancelado**.
- Actualizar la disponibilidad del vehículo de acuerdo con el estado de su mantenimiento.
- Registrar información relacionada con la intervención realizada, como **descripción, kilometraje, costo y fecha de finalización**.
- Consultar el **historial de mantenimientos de cada vehículo**.

Además, la base de datos debe estar poblada previamente (seeders) para poder realizar las validaciones y pruebas necesarias.

Funcionalidades principales.

Para alcanzar un resultado óptimo en esta prueba, deberás cumplir cada uno de los siguientes requisitos:

Requisitos:

1. Sistema de autenticación y autorización

1. La aplicación deberá permitir el **registro de usuarios** con los siguientes roles:
    a. ADMIN
    b. TECHNICIAN
2. La aplicación deberá permitir el **inicio de sesión** mediante correo electrónico y contraseña.
3. Las contraseñas deberán almacenarse de forma segura utilizando un mecanismo de hashing.
4. Después de una autenticación exitosa, la aplicación deberá generar un **JSON Web Token (JWT)**.
5. Las rutas privadas deberán estar protegidas mediante **Guards de NestJS**.
6. Se deberá implementar autorización basada en roles utilizando:
    a. un decorador personalizado para declarar roles permitidos;
    b. un RolesGuard encargado de validar los permisos del usuario.
7. El sistema deberá implementar como mínimo tres **decoradores personalizado** relacionado con autenticación o autorización. Por ejemplo:
    a. @Roles()

b. @CurrentUser()
c. @Public()

8. Se deberá implementar autenticación mediante **x-api-key** para al menos un endpoint destinado a integraciones externas. Por ejemplo:



```json
{
  "success": true,
  "statusCode": 200,
  "data": {
    "vehicle": {
      "id": 1,
      "plate": "ABC123",
      "brand": "Chevrolet",
      "model": "NPR",
      "available": true
    },
    "maintenances": [
      {
        "id": 12,
        "type": "PREVENTIVO",
        "status": "COMPLETADO",
        "description": "Cambio de aceite y filtros",
        "mileage": 45000,
        "cost": 350000,
        "finishedAt": "2026-09-15T16:30:00.000Z",
        "mechanic": { "id": 3, "name": "Carlos Pérez" },
        "workshop": { "id": 2, "name": "Taller Norte" }
      },
      {
        "id": 18,
        "type": "CORRECTIVO",
        "status": "COMPLETADO",
        "description": "Reemplazo de pastillas de freno",
        "mileage": 52300,
        "cost": 480000,
        "finishedAt": "2026-10-10T14:00:00.000Z",
        "mechanic": { "id": 5, "name": "Laura Gómez" },
        "workshop": { "id": 1, "name": "Taller Centro" }
      }
    ]
  },
  "timestamp": "2026-10-20T19:00:00.000Z"
}
```

9. La API Key deberá almacenarse mediante variables de entorno y deberá validarse mediante un Guard específico.
10. No se permitirá el acceso a información protegida cuando:
    a. no exista token;
    b. el token sea inválido o haya expirado;
    c. el usuario no posea el rol requerido;

d. la API Key sea incorrecta.

Criterios de aceptación:

1. Funcionalidad completa:

1. Los usuarios deberán poder registrarse e iniciar sesión correctamente.
2. El sistema deberá manejar como mínimo los siguientes roles:
    a. **Administrador:** podrá gestionar vehículos, mecánicos, talleres y mantenimientos, además de consultar historiales y auditorías.
    b. **Técnico:** podrá consultar los mantenimientos que tenga asignados, y visualizar su información y el taller donde debe realizar cad auno y actualizar su estado de acuerdo con las reglas establecidas.

3. Los usuarios autenticados solo podrán ejecutar las operaciones autorizadas para su rol.
4. La API deberá responder utilizando los códigos HTTP apropiados para cada operación.
5. Todas las funcionalidades principales deberán estar disponibles mediante endpoints REST correctamente estructurados.
6. La aplicación deberá iniciar y operar correctamente utilizando las variables de entorno definidas.

2. Gestor de vehículos

1. Se deberá exponer un endpoint protegido que permita listar los vehículos registrados.
2. El administrador deberá poder registrar nuevos vehículos.
3. Cada vehículo deberá contener como mínimo:
    a. placa;
    b. marca;
    c. modelo;
    d. año;
    e. kilometraje;
    f. estado.
4. No se podrán registrar dos vehículos con la misma placa.
5. Se deberá permitir consultar la información de un vehículo mediante su identificador o placa.
6. El administrador podrá actualizar la información de un vehículo.
7. Se deberá implementar eliminación lógica o inactivación cuando corresponda, evitando eliminar información relacionada con mantenimientos históricos.

8. Los estados mínimos del vehículo serán:
    a. AVAILABLE
    b. IN_MAINTENANCE
    c. INACTIVE
9. Un vehículo en estado INACTIVE no podrá ser asociado a un nuevo mantenimiento.

3. Gestor de mecánicos

1. El administrador deberá poder registrar mecánicos.
2. Cada mecánico deberá contener como mínimo:
    a. nombre;
    b. correo;
    c. especialidad;
    d. estado.
3. Se deberá permitir consultar todos los mecánicos registrados.
4. Se deberá permitir consultar un mecánico específico.
5. El administrador podrá actualizar la información de un mecánico.
6. Se deberá permitir activar e inactivar mecánicos.
7. Un mecánico inactivo no podrá ser asignado a nuevos mantenimientos.
8. La inactivación de un mecánico no deberá eliminar su historial de mantenimientos realizados.
9. Un mecánico no está vinculado de forma permanente a un taller. Puede realizar mantenimientos en distintos talleres en diferentes horarios.
10. Un mecánico no podrá tener dos mantenimientos activos (SCHEDULED o IN_PROGRESS) cuyos horarios se crucen, sin importar si son en el mismo taller o en talleres diferentes. Por ejemplo:
    a. No podrá tener dos mantenimientos a las 7:30 en el mismo taller.
    b. No podrá tener un mantenimiento a las 7:30 en un taller y otro a las 7:30 en un taller diferente.
    c. Si tiene un mantenimiento de 7:30 a 9:00, tampoco podrá asignársele otro que inicie a las 8:00, en ningún taller.
    d. Sí podrá tener un mantenimiento de 7:30 a 9:00 y otro que inicie a las 9:00, en el mismo taller o en otro.

4. Gestor de talleres

1. El administrador deberá poder registrar talleres.
2. Cada taller deberá contener como mínimo:
    a. nombre;



   b. dirección;
   c. estado.
3. Se deberá exponer un endpoint protegido que permita listar los talleres activos.
4. Se deberá permitir consultar la información de un taller específico.
5. El administrador podrá activar o inactivar talleres.
6. No se podrá asignar un mantenimiento a un taller inexistente o inactivo.

5. Gestor de mantenimientos

1. Se deberá exponer un endpoint protegido que permita crear mantenimientos.
2. Cada mantenimiento deberá estar relacionado con:
   a. un vehículo;
   b. un mecánico;
   c. un taller.
3. El mantenimiento deberá contener como mínimo:
   a. tipo;
   b. descripción;
   c. kilometraje;
   d. fecha programada;
   e. estado;
   f. vehículo;
   g. mecánico;
   h. taller.
4. Los tipos permitidos serán:
   a. PREVENTIVE
   b. CORRECTIVE
5. El estado inicial de un nuevo mantenimiento deberá ser:
   a. SCHEDULED
6. Los estados permitidos serán:
   a. SCHEDULED
   b. IN_PROGRESS
   c. COMPLETED
   d. CANCELLED

7. Se deberá exponer un endpoint que permita actualizar el estado de un mantenimiento existente.
8. Se deberá permitir consultar:
    a. todos los mantenimientos;
    b. mantenimientos activos;
    c. mantenimientos por vehículo;
    d. mantenimientos asignados a un mecánico;
    e. historial de mantenimientos de un vehículo.
9. Para finalizar un mantenimiento deberán registrarse como mínimo:
    a. trabajo realizado;
    b. costo final;
    c. fecha de finalización;
    d. kilometraje final.

6. Reglas de negocio

- No se podrá crear un mantenimiento para un vehículo inexistente.
- No se podrá asignar un mecánico inexistente o inactivo.
- No se podrá asignar un taller inexistente o inactivo.
- Un vehículo no podrá tener simultáneamente más de un mantenimiento activo.
- Un tecnico no podrá tener simultáneamente más de un mantenimiento activo.
- El kilometraje registrado en un mantenimiento no podrá ser inferior al kilometraje actual del vehículo.
- Cuando un mantenimiento cambie a **IN_PROGRESS** el vehículo deberá cambiar automáticamente a **IN_MAINTENANCE**
- Cuando el mantenimiento cambie correctamente a **COMPLETED** el vehículo deberá quedar nuevamente **AVAILABLE.**
- Un mantenimiento **COMPLETED** no podrá regresar a un estado anterior.
- Un mantenimiento **CANCELLED** no podrá posteriormente cambiar a **IN_PROGRESS** o **COMPLETED**.
- No se podrá completar un mantenimiento si no contiene la información obligatoria de cierre.

- Las transiciones de estado inválidas deberán generar una respuesta HTTP apropiada y un mensaje descriptivo.

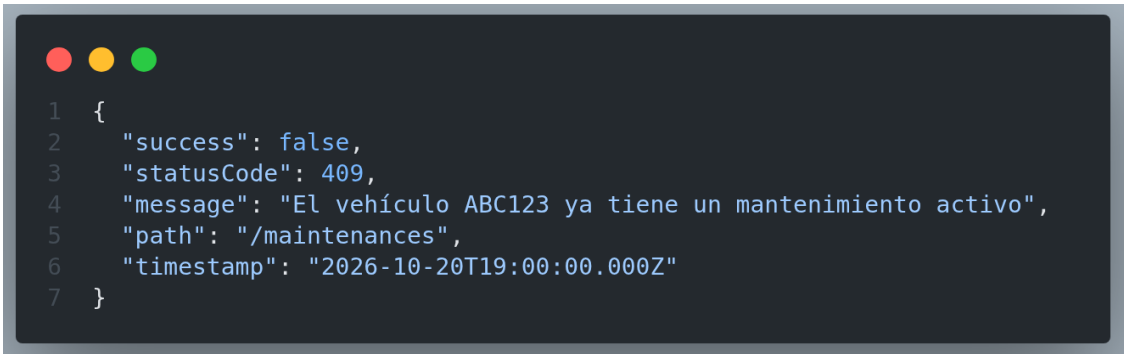
7. Respuesta y Errores

1. Las respuestas exitosas deberán tener un formato uniforme, aplicado por el interceptor. Por ejemplo:



```json
{
  "success": true,
  "statusCode": 200,
  "data": { },
  "timestamp": "2026-10-20T19:00:00.000Z"
}
```

2. Los errores deberán ser procesados por el Exception Filter personalizado e incluir como mínimo el código HTTP, el mensaje, la ruta y la fecha del error. Por ejemplo:



```json
{
  "success": false,
  "statusCode": 409,
  "message": "El vehículo ABC123 ya tiene un mantenimiento activo",
  "path": "/maintenances",
  "timestamp": "2026-10-20T19:00:00.000Z"
}
```

3. El formato exacto puede variar, pero deberá mantenerse consistente en todos los endpoints.
4. Los errores de validación, autenticación, autorización, recursos inexistentes y conflictos de negocio (incluidos los de unicidad de la base de datos) **no deberán convertirse en errores HTTP 500**.

8. Persistencia relacional con TypeORM

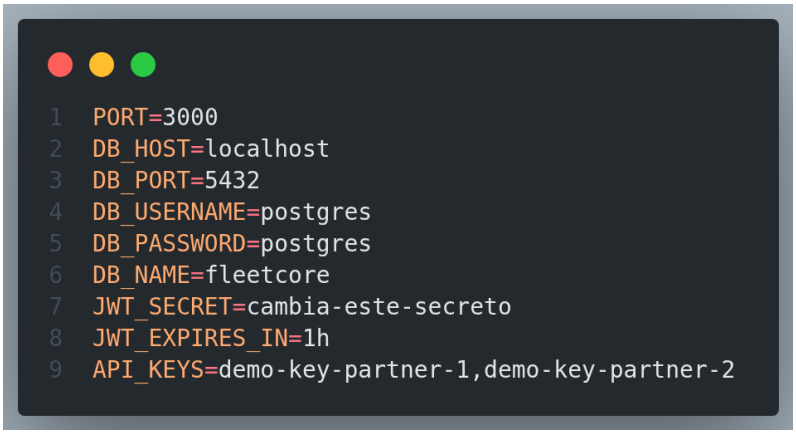
1. PostgreSQL deberá utilizarse como base de datos relacional principal.

2. La integración deberá realizarse mediante **TypeORM**.

3. Deberán existir relaciones correctamente definidas entre las entidades.

9. Seeders y variables de entorno

1. La base de datos deberá poblarse con un solo comando (por ejemplo, npm run seed) y como mínimo con:
   a. 1 usuario **ADMIN** y 2 usuarios **TECHNICIAN**;
   b. 3 mecánicos: 2 activos vinculados a los técnicos y 1 inactivo;
   c. 2 talleres: 1 activo y 1 inactivo;
   d. 4 vehículos: 2 **AVAILABLE**, 1 **IN_MAINTENANCE** (con su mantenimiento **IN_PROGRESS**) y 1 **INACTIVE**;
   e. al menos 1 mantenimiento **COMPLETED** para mostrar historial.
2. Ejecutar el seeder dos veces no deberá duplicar registros.
3. Se deberá entregar un archivo .env.example con claves ficticias, por ejemplo:



```
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=postgres
DB_PASSWORD=postgres
DB_NAME=fleetcore
JWT_SECRET=cambia-este-secreto
JWT_EXPIRES_IN=1h
API_KEYS=demo-key-partner-1,demo-key-partner-2
```

10. Clean code, entrega y lógica de rutas

1. El código deberá contar con buenas prácticas de escritura y nombres claros.
2. Los archivos deberán estar organizados y presentes en el repositorio del proyecto.
3. El código deberá contener comentarios claros en las secciones clave (**guards**, **filtro**, **interceptor**, **transacciones**).
4. El repositorio deberá evidenciar **commits** descriptivos por funcionalidad.
5. Se deberá indicar en el **README** la URL para visualizar la documentación en **Swagger**.
6. Solo los usuarios autenticados podrán acceder a los endpoints, y las rutas deberán estar protegidas según el rol del usuario.

Entregables

1. Enlace al repositorio en **GitHub** (público).

2. Archivo **README** con la información del coder (**nombre y clan**) y el paso a paso para levantar y usar la solución: instalación, variables de entorno, ejecución del seeder, credenciales de prueba y URL de Swagger.
3. Archivo .env.example.
4. Dump o archivo **.sql** cargado en Moodle con la estructura y los datos utilizados.
5. Documentación en **Swagger** que permita probar la solución.
6. Evidencias de pruebas de los escenarios principales (colección de Postman o capturas de Swagger).
   Fecha y hora de entrega: **por definir**.

Puntos extra (opcionales)

Se entregará un máximo de **20 puntos** en total, aplicables a la rúbrica de **Funcionalidad y validaciones**. Solo se califican si los requisitos obligatorios funcionan.

1. **Despliegue con Docker (5 puntos):** Dockerfile de la API y docker-compose.yml que levante la API y PostgreSQL con volumen para persistencia y red interna entre servicios.

2. **Pruebas unitarias con Jest (5 puntos):** al menos dos pruebas sobre funcionalidades clave (por ejemplo, las reglas de cambio de estado), con cobertura superior al 40 % verificable con --coverage.

3. **OAuth 2.0 (10 puntos):** autenticación con un proveedor externo (Google o GitHub) que integre al usuario a la API y le entregue un JWT válido.

Recursos

- Documentación oficial de NestJS
- Documentación oficial de TypeScript
- Documentación oficial de TypeORM
- Documentación oficial de PostgreSQL
- Documentación de JSON Web Token
- OpenAPI (Swagger) en NestJS